PROJECT REPORT

Implement Client Script & UI Policy – Incident

1. Introduction

ServiceNow Incident Management requires accurate and complete information for effective incident handling. Manual data entry can result in missing or incorrect information, which may affect reporting, SLA compliance, and service quality. This project demonstrates how UI Policies and Client Scripts can be used in ServiceNow to control Incident form behavior, validate information, automatically update fields, and prevent incorrect submissions.

2. Problem Statement

Incident records may contain incomplete or incorrect information when users enter data manually. Therefore, there is a need for automated validation and conditional field control at the user-interface level. The project addresses this problem by using UI Policies and Client Scripts to make fields mandatory, automatically set values, control field editing, validate data before saving, and prevent unauthorized list-based updates.

3. Objectives

• Implement UI Policies for Incident records.
• Make required fields mandatory for high-impact incidents.
• Automatically set Urgency to High when Impact is High.
• Make the Urgency field read-only for high-impact incidents.
• Prevent saving when required information is missing.
• Prevent direct State changes through list editing.
• Test and verify all configured behaviors.

4. Technologies Used

• ServiceNow
• Incident Management
• UI Policy
• UI Policy Actions
• Client Scripts
• JavaScript
• Form Validation

5. Implementation

5.1 UI Policy – High Impact Control

A UI Policy named “High Impact Control” is created for the Incident table. The condition is Impact = 1 – High. When the condition is met, additional controls are applied. Reverse if false is enabled so that the changes are reversed when the condition is no longer true.

5.2 UI Policy Action – Urgency

An action is created for the Urgency field. When Impact is High, the Urgency field remains visible but becomes read-only, preventing users from changing its value.

5.3 onChange Client Script

The “Auto set urgency for high impact” Client Script runs when Impact changes. If Impact is High, Urgency is automatically set to High and an information message is displayed.

5.4 onSubmit Client Script

The “Prevent save if Assigned To missing” Client Script validates the Incident before saving. If Impact is High and Assigned To is empty, saving is prevented and an error message is displayed.

5.5 onCellEdit Client Script

The onCellEdit Client Script prevents users from changing the State field directly from the Incident list. An alert is displayed and the change is blocked.

6. Testing

The configuration is tested using different scenarios. When Impact is set to High, the UI Policy and Client Script behaviors are triggered. If Assigned To is empty, the Incident cannot be saved. When Assigned To is filled, the Incident can be saved. Changing Impact from High to Medium reverses the mandatory and read-only behavior. Direct State editing from the list is blocked, while changing State through the Incident form is allowed.

7. Results

The implementation demonstrates conditional field behavior, mandatory field enforcement, automatic value population, read-only field control, save-time validation, list-edit restrictions, and reversal of UI Policy conditions. The tests confirm that the configured UI Policies and Client Scripts work as intended.

8. Advantages

1. Improves data accuracy.
2. Reduces incomplete Incident records.
3. Automates repetitive field updates.
4. Prevents invalid data submission.
5. Improves user interaction with Incident forms.
6. Provides consistent business-rule enforcement.
7. Lightweight and easy to implement.

9. Conclusion

The Implement Client Script & UI Policy (Incident) project demonstrates how ServiceNow can use UI Policies and Client Scripts together to improve Incident data quality and user-interface behavior. The project uses onChange, onSubmit, and onCellEdit Client Scripts along with UI Policy Actions to control fields such as Impact, Assigned To, Urgency, and State. The implementation helps maintain consistent and valid Incident records while providing an efficient and practical solution for ServiceNow micro-project development.

Test Case Summary

Test Case	Expected Result
Impact set to High	UI Policy and Client Script are triggered
Assigned To left empty	Incident cannot be saved

Test Case	Expected Result
Assigned To filled	Incident saves successfully
Impact changed High → Medium	Mandatory/read-only behavior is reversed
State edited from list	Change is blocked
State changed from form	Change is allowed